# Spring Data JPA

## Repository Abstraction, Query Methods, Sorting and Pagination

Spring Data JPA simplifies the repository layer of a JPA application. It does not replace JPA or Hibernate. Instead, it builds a higher-level repository abstraction on top of them.

```
Application service
        ↓
Spring Data JPA repository
        ↓
JPA EntityManager
        ↓
Hibernate
        ↓
JDBC driver
        ↓
Relational database
```

---

# 1. The Persistence Layer after Hibernate and JPA

Before JPA and Hibernate, persistence code commonly used JDBC or Spring JDBC:

```
public Student findById(Long id) {
    String sql = """
            SELECT id, name, email
            FROM students
            WHERE id = ?
            """;

    return jdbcTemplate.queryForObject(
```

```
            sql,
            studentRowMapper,
            id
    );
}
```

This code requires the developer to handle:

- SQL statements
- Parameter binding
- Result-set mapping
- Object creation
- Database-specific details

JPA and Hibernate remove much of this work:

```
public Student findById(Long id) {
    return entityManager.find(Student.class, id);
}
```

JPA and Hibernate provide:

- Object-relational mapping
- Entity state management
- Persistence context
- Dirty checking
- Relationship mapping
- SQL generation
- First-level caching
- Transaction synchronization

However, an application still needs a dedicated persistence layer. Business services should focus on use cases, while repositories should contain data-access logic.

```java
@Repository
public class StudentRepository {

    @PersistenceContext
    private EntityManager entityManager;

    public void save(Student student) {
        entityManager.persist(student);
    }

    public Student findById(Long id) {
        return entityManager.find(Student.class, id);
    }

    public void delete(Student student) {
        entityManager.remove(student);
    }
}
```

> JPA and Hibernate do not eliminate the repository layer. They simplify the persistence logic written inside it.

---

# 2. The Repository Boilerplate Problem

Suppose an application contains three entities:

```
Student
Department
Course
```

Without Spring Data JPA, each entity may need its own repository implementation:

```
StudentRepository
DepartmentRepository
```

```
CourseRepository
```

These repositories repeatedly implement the same basic operations:

- Save an entity
- Find an entity by ID
- Find all entities
- Check whether an entity exists
- Count entities
- Delete an entity

Only two pieces of information usually change:

1. The entity type
2. The identifier type

This common structure can be represented generically:

```
Repository<EntityType, IdentifierType>
```

Examples:

```
Repository<Student, Long>
Repository<Department, Long>
Repository<Course, Integer>
```

This generic repository idea is the foundation of Spring Data.

---

# 3. Spring Data and Spring Data JPA

## Spring Data

Spring Data is an umbrella project that provides a consistent repository-based programming model for different persistence technologies.

Examples include:

- Spring Data JPA
- Spring Data JDBC
- Spring Data MongoDB
- Spring Data Redis
- Spring Data Elasticsearch
- Spring Data Cassandra
- Spring Data Neo4j

The central idea is simple:

> Define a repository interface for a domain type, and let the appropriate Spring Data module provide the supporting implementation and infrastructure.

The implementation underneath depends on the selected module.

```
Spring Data JPA repository
        ↓
EntityManager
        ↓
JPA provider such as Hibernate
        ↓
Relational database
```

```
Spring Data MongoDB repository
        ↓
MongoDB infrastructure
        ↓
MongoDB
```

## Spring Data JPA

Spring Data JPA is the Spring Data module designed for applications that use the Jakarta Persistence API.

Instead of manually implementing a repository class:

```
@Repository
public class StudentRepository {

    @PersistenceContext
    private EntityManager entityManager;

    public Student findById(Long id) {
        return entityManager.find(Student.class, id);
    }
}
```

the developer can declare an interface:

```
public interface StudentRepository
        extends JpaRepository<Student, Long> {
}
```

The repository can then be injected and used like a normal Spring bean:

```
@Service
public class StudentService {

    private final StudentRepository studentRepository;

    public StudentService(StudentRepository studentRepositor
y) {
        this.studentRepository = studentRepository;
    }
}
```

Common operations become immediately available:

```
studentRepository.save(student);
studentRepository.findById(id);
```

```
studentRepository.findAll();
studentRepository.deleteById(id);
```

At startup, Spring Data detects the interface and creates a repository bean for it.



# 4. JPA versus Spring Data JPA

| JPA | Spring Data JPA |
|---|---|
| Jakarta specification | Spring framework module |
| Defines persistence concepts and APIs | Builds a repository abstraction on top of JPA |
| Provides `EntityManager` and JPQL | Provides repository interfaces and query-method support |
| Requires a persistence provider | Uses the configured JPA provider underneath |
| Does not contain `JpaRepository` | Provides `JpaRepository` |

## JPA provides

```
@Entity
@Id
@OneToMany
@ManyToOne
EntityManager
EntityManagerFactory
TypedQuery
```

Typical JPA operations are:

```
entityManager.persist(student);
entityManager.find(Student.class, 1L);
entityManager.remove(student);
```

## Spring Data JPA provides

```
Repository
CrudRepository
ListCrudRepository
PagingAndSortingRepository
JpaRepository
@Query
@NativeQuery
@Modifying
@EntityGraph
JpaSpecificationExecutor
```

> `JpaRepository` is part of Spring Data JPA, not part of the JPA specification.

# Main responsibilities of Spring Data JPA

## Common CRUD operations

```
save()
findById()
findAll()
existsById()
count()
deleteById()
```

## Runtime repository implementation

```
public interface StudentRepository
        extends JpaRepository<Student, Long> {
}
```

Spring Data JPA creates the repository bean; the developer does not write the implementation class.

## Query derivation from method names

```
List<Student> findByDepartmentName(String departmentName);
```

### Custom JPQL and native SQL

```
@Query("""
       SELECT s
       FROM Student s
       WHERE s.email = :email
       """)
Optional<Student> findStudentByEmail(
        @Param("email") String email
);
```

### Sorting and pagination

```
Page<Student> findAll(Pageable pageable);
```

## What Spring Data JPA does not do

Spring Data JPA does not remove the need to understand:

- Entity mapping
- Transactions
- Managed and detached entity states
- Lazy loading and fetching
- SQL performance
- Database indexes and constraints
- The N+1 query problem

It reduces repository boilerplate; it does not replace sound persistence design.

# 5. Repository Interface Hierarchy

The important repository abstractions are:

```
Repository<T, ID>
├── CrudRepository<T, ID>
│   └── ListCrudRepository<T, ID>
└── PagingAndSortingRepository<T, ID>
    └── ListPagingAndSortingRepository<T, ID>

JpaRepository<T, ID>
├── ListCrudRepository<T, ID>
├── ListPagingAndSortingRepository<T, ID>
└── QueryByExampleExecutor<T>
```

## `Repository<T, ID>`

```java
public interface Repository<T, ID> {
}
```

`Repository` is primarily a marker interface. It identifies:

- The domain type: `T`
- The identifier type: `ID`
- The interface as a Spring Data repository

Extending it does not automatically expose the full CRUD API. This allows a restricted repository contract:

```java
public interface StudentRepository
        extends Repository<Student, Long> {

    Student save(Student student);

    Optional<Student> findById(Long id);
```

```
    List<Student> findAll();
}
```

Because no delete method is declared, callers cannot delete students through this repository interface.

## `CrudRepository<T, ID>`

`CrudRepository` extends `Repository` and provides generic CRUD operations:

```
public interface CrudRepository<T, ID>
        extends Repository<T, ID> {

    <S extends T> S save(S entity);

    Optional<T> findById(ID id);

    Iterable<T> findAll();

    boolean existsById(ID id);

    long count();

    void deleteById(ID id);

    void delete(T entity);
}
```

## `ListCrudRepository<T, ID>`

`ListCrudRepository` offers CRUD functionality similar to `CrudRepository`, but methods returning multiple entities return a `List` instead of an `Iterable`.

```
public interface StudentRepository
        extends ListCrudRepository<Student, Long> {
}
```

```
List<Student> students = studentRepository.findAll();
```

## `PagingAndSortingRepository<T, ID>`

This interface adds sorted and paginated retrieval:

```
Iterable<T> findAll(Sort sort);

Page<T> findAll(Pageable pageable);
```

Since Spring Data 3, the sorting repository hierarchy is separate from the CRUD hierarchy. Extending only `PagingAndSortingRepository` does not guarantee methods such as `save()` and `deleteById()`.

To combine the lower-level contracts explicitly:

```
public interface StudentRepository
        extends ListCrudRepository<Student, Long>,
                ListPagingAndSortingRepository<Student, Long>
{
}
```

In a JPA application, extending `JpaRepository` is normally more convenient because it already combines these capabilities.

## `JpaRepository<T, ID>`

```
public interface StudentRepository
        extends JpaRepository<Student, Long> {
}
```

It provides three broad groups of operations.

### CRUD operations

```
save()
saveAll()
findById()
findAll()
existsById()
count()
delete()
deleteById()
deleteAll()
```

### Sorting and pagination

```
findAll(Sort sort)
findAll(Pageable pageable)
```

### JPA-specific operations

```
flush()
saveAndFlush()
saveAllAndFlush()
getReferenceById()
deleteAllInBatch()
deleteAllByIdInBatch()
```

> Batch delete methods can behave differently from deleting managed entities one by one. They should be used deliberately, especially when cascades, lifecycle callbacks, or the current persistence context matter.

---

# 6. Declaring a Repository Interface

A typical repository declaration is:

```java
package com.coderarmy.jpa.repository;

import com.coderarmy.jpa.model.Student;
import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository
        extends JpaRepository<Student, Long> {
}
```

Normally:

- No implementation class is required.
- The interface does not need `@Repository`.
- Spring Boot scans repository interfaces under the application's base package.
- `@EnableJpaRepositories` is generally unnecessary in a standard Spring Boot project, but it can be used for explicit or custom repository scanning.

The interface can declare additional query methods:

```java
public interface StudentRepository
        extends JpaRepository<Student, Long> {

    Optional<Student> findByEmail(String email);

    List<Student> findByAgeGreaterThan(Integer age);
}
```

Repository methods fall into two main categories:

| Category | Examples | How they are handled |
| --- | --- | --- |
| Inherited methods | `save()`, `findById()`, `findAll()` | Implemented by the repository base class |
| Declared query methods | `findByEmail()`, `findByAgeGreaterThan()` | Converted into or associated with queries |

# 7. Repository Proxy Internals

## Why an interface can be injected

The following interface cannot be instantiated directly:

```
public interface StudentRepository
        extends JpaRepository<Student, Long> {
}
```

```
new StudentRepository(); // compilation error
```

Spring Data creates a runtime object that implements the repository interface. This runtime-generated object is called a repository proxy.

```
StudentService
      ↓
StudentRepository proxy
      ↓
Classifies the invoked method
      ↓
Chooses the correct execution path
```

For an inherited method:

```
studentRepository.findById(1L)
      ↓
Repository proxy
      ↓
SimpleJpaRepository.findById(1L)
      ↓
EntityManager.find(...)
```

For a derived method:

```
studentRepository.findByEmail(email)
      ↓
Repository proxy
```

```
      ↓
Prepared query-method execution
      ↓
Bind parameter
      ↓
EntityManager executes query
```

## `SimpleJpaRepository`

`SimpleJpaRepository<T, ID>` is the standard base implementation behind most `JpaRepository` operations.

Conceptually, it contains:

```
public class SimpleJpaRepository<T, ID>
        implements JpaRepository<T, ID> {

    private final EntityManager entityManager;
    private final JpaEntityInformation<T, ?> entityInformatio
n;
}
```

It implements methods such as:

```
save()
findById()
findAll()
existsById()
count()
delete()
flush()
saveAndFlush()
getReferenceById()
```

# Why the proxy is still necessary

`SimpleJpaRepository` knows how to execute standard repository operations, but it cannot contain every application-specific method:

```
public interface StudentRepository
        extends JpaRepository<Student, Long> {

    Optional<Student> findByEmail(String email);

    List<Student> findByDepartmentName(String name);
}
```

The proxy combines multiple sources of behavior:

| Method type | Execution source |
| --- | --- |
| Standard CRUD method | `SimpleJpaRepository` |
| Derived query method | Query created from the method name |
| `@Query` method | Declared JPQL or SQL query |
| Default interface method | Java interface default method |
| Custom repository fragment | Developer-provided implementation |

Spring Data JPA remains a layer above JPA. Its base implementation ultimately delegates persistence work to `EntityManager`, whose implementation is commonly provided by Hibernate.

---

# 8. Important `JpaRepository` Methods

## `save()`

```
Student savedStudent = studentRepository.save(student);
```

`save()` does not simply mean “execute an `INSERT`.” It means that the entity state should become persistent.

Conceptually:

```
Is the entity new?
      ↓
```

```
Yes → EntityManager.persist(entity)
No  → EntityManager.merge(entity)
```

Spring Data generally determines whether an entity is new by inspecting its version property or identifier. An entity can also implement `Persistable` to control this decision.

The return value matters when `merge()` is used:

```
Student managedStudent = studentRepository.save(detachedStude
nt);
```

`merge()` returns a managed instance. The detached object passed as the argument does not automatically become managed.

## saveAll()

```
List<Student> savedStudents = studentRepository.saveAll(
        List.of(student1, student2, student3)
);
```

Conceptually, the standard implementation calls `save()` for each entity:

```
for (Student student : students) {
    result.add(save(student));
}
```

`saveAll()` does not itself guarantee one multi-row SQL statement. Hibernate/JDBC batching is a separate optimization that requires suitable configuration and an identifier strategy that supports it.

## findById()

```
Optional<Student> student = studentRepository.findById(1L);
```

The core lookup delegates to `EntityManager.find()`.

```
findById(1L)
      ↓
Check persistence context
      ↓
Already managed?
├── Yes → return the managed instance
└── No  → query the database if necessary
```

It returns `Optional<Student>` because a row with the requested ID may not exist.

## `findAll()`

```java
List<Student> students = studentRepository.findAll();
```

The method means “load every `Student` matched by the repository operation.” It is suitable for small datasets but dangerous for large tables. Spring Data cannot infer that pagination was intended.

## `existsById()`

```java
boolean exists = studentRepository.existsById(1L);
```

This expresses an existence question rather than a request to load the complete entity. Spring Data generates an appropriate existence query, commonly using a count or equivalent mechanism.

## `count()`

```java
long numberOfStudents = studentRepository.count();
```

Conceptually:

```sql
SELECT COUNT(*)
FROM students;
```

## `delete()`

```
studentRepository.delete(student);
```

Deletion must ultimately operate on a managed entity. If the supplied entity is detached, the repository implementation may obtain or merge the corresponding managed state before removing it.

## `deleteById()`

```
studentRepository.deleteById(id);
```

Use this when the identifier is available and the repository should remove the corresponding entity.

## `deleteAll()`

```
studentRepository.deleteAll();
```

This is a broad and potentially expensive operation. It may process entities individually. For bulk deletion, JPA-specific batch methods or an explicit bulk query may be more suitable, but their persistence-context and lifecycle behavior must be understood.

## `flush()`

```
studentRepository.flush();
```

`flush()` synchronizes pending changes in the persistence context with the database. It does not commit the transaction.

```
flush  = send pending SQL to the database
commit = permanently complete the transaction
```

The transaction can still roll back after a flush.

## `saveAndFlush()`

```
Student savedStudent = studentRepository.saveAndFlush(studen
t);
```

Conceptually:

```
Student result = save(student);
flush();
return result;
```

Use it when the database must be synchronized immediately—for example, to detect a constraint violation before the transaction reaches its normal commit point. It should not be used automatically for every save.

---

## 9. `save()` versus Dirty Checking

Consider a managed entity loaded inside a transaction:

```
@Transactional
public void renameStudent(Long id, String newName) {
    Student student = studentRepository.findById(id)
            .orElseThrow();

    student.setName(newName);
}
```

No call to `save()` is required for JPA to update the row.

The process is:

```
Repository loads Student
        ↓
Student becomes managed
        ↓
Application changes the managed object
        ↓
Dirty checking detects the change
```

```
        ↓
UPDATE is generated during flush/commit
```

Calling `save()` for the managed entity is usually redundant from a pure JPA perspective. It may still be used when a team deliberately follows the repository abstraction consistently, but it is not what causes dirty checking.

`save()` is important when:

- Persisting a new entity
- Reattaching state from a detached entity through `merge()`
- Returning the managed instance produced by `merge()`

---

# 10. Derived Query Methods

## Query derivation

Without query derivation:

```
@Query("""
       SELECT s
       FROM Student s
       WHERE s.email = :email
       """)
Optional<Student> findByEmail(@Param("email") String email);
```

With query derivation:

```
Optional<Student> findByEmail(String email);
```

Spring Data parses the method name and creates a query from it.

```
find  → selection operation
By    → beginning of the predicate
Email → Student.email property
```

Conceptual JPQL:

```
SELECT s
FROM Student s
WHERE s.email = ?1
```

The default lookup strategy first checks for a declared query and derives a query from the method name if none is found.

## Subject and predicate

```
findByEmail
```

contains two logical sections:

```
findBy  → subject
Email   → predicate
```

The subject describes the operation:

```
findByEmail(...)
existsByEmail(...)
countByActiveTrue()
deleteByActiveFalse()
findTop5ByActiveTrueOrderByAgeDesc()
```

The predicate describes the matching condition:

```
findByAgeGreaterThanAndActiveTrue(Integer age)
```

Conceptual JPQL:

```
SELECT s
FROM Student s
```

```
WHERE s.age > ?1
  AND s.active = true
```

## Common query keywords

| Keyword | Repository method | Meaning |
|---|---|---|
| Equality | `findByEmail(String email)` | `email = ?` |
| `And` | `findByNameAndAge(String name, Integer age)` | Both conditions must match |
| `Or` | `findByNameOrEmail(String name, String email)` | Either condition may match |
| `GreaterThan` | `findByAgeGreaterThan(Integer age)` | `age > ?` |
| `LessThan` | `findByAgeLessThan(Integer age)` | `age < ?` |
| `Between` | `findByAgeBetween(Integer min, Integer max)` | Inclusive range |
| `Like` | `findByNameLike(String pattern)` | Caller supplies the pattern |
| `Containing` | `findByNameContaining(String value)` | Contains the supplied value |
| `StartingWith` | `findByNameStartingWith(String prefix)` | Starts with the value |
| `EndingWith` | `findByNameEndingWith(String suffix)` | Ends with the value |
| `In` | `findByAgeIn(Collection<Integer> ages)` | Value belongs to the collection |
| `NotIn` | `findByAgeNotIn(Collection<Integer> ages)` | Value does not belong to the collection |
| `IsNull` | `findByDepartmentIsNull()` | Property is `null` |
| `IsNotNull` | `findByDepartmentIsNotNull()` | Property is not `null` |
| `True` | `findByActiveTrue()` | Boolean property is true |
| `False` | `findByActiveFalse()` | Boolean property is false |
| `IgnoreCase` | `findByNameIgnoreCase(String name)` | Case-insensitive comparison |
| `OrderBy` | `findByActiveTrueOrderByNameAsc()` | Static ordering |
| `Top` / `First` | `findTop5ByOrderByAgeDesc()` | Limit the result |

### `Like` versus `Containing`

With `Like`, the caller supplies the wildcard characters:

```
List<Student> findByNameLike(String pattern);

studentRepository.findByNameLike("%har%");
```

With `Containing`, Spring Data adds the contains pattern:

```
List<Student> findByNameContaining(String value);

studentRepository.findByNameContaining("har");
```

Conceptually, the second method binds a pattern such as `%har%`.

### Limiting results

```
List<Student> findTop5ByActiveTrueOrderByAgeDesc();
```

This means:

1. Select active students.
2. Sort them by age in descending order.
3. Return at most five.

An explicit order is important. Without it, “top five” has no reliable business meaning because a relational database does not guarantee a natural row order.

```
Optional<Student> findFirstByDepartmentNameOrderByAgeDesc(
        String departmentName
);
```

## Limitations of derived query methods

Derived queries are ideal when the method name remains short and readable:

```
findByEmail(...)
findByActiveTrueOrderByNameAsc()
```

They become difficult to maintain when they grow into long query-language sentences:

```
findTop10ByNameContainingIgnoreCaseAndAgeGreaterThanAndActive
TrueAndDepartmentNameOrderByAdmissionDateDesc(...)
```

Main limitations:

- Long names are difficult to read.
- Complex `And` / `Or` grouping is hard to express clearly.
- The method name cannot show complicated joins and projections well.
- Database-specific behavior may require native SQL.

For such cases, use `@Query`, specifications, Query by Example, Querydsl, or a custom repository implementation depending on the problem.

---

# 11. Custom JPQL Queries

## `@Query`

`@Query` keeps a custom query close to the repository method that executes it.

```
@Query("""
       SELECT s
       FROM Student s
       WHERE s.email = :email
       """)
Optional<Student> findStudentByEmail(
        @Param("email") String email
);
```

JPQL queries:

- Entity names
- Entity fields
- Entity relationships

JPQL does not directly use table names and database column names.

```
@Entity
public class Student {
}
```

```
SELECT s
FROM Student s
```

`Student` is the entity name; `s` is its query alias.

## Named parameters

```
@Query("""
        SELECT s
        FROM Student s
        WHERE s.age >= :minimumAge
          AND s.age <= :maximumAge
        """)
List<Student> findStudentsInAgeRange(
        @Param("minimumAge") Integer minimumAge,
        @Param("maximumAge") Integer maximumAge
);
```

Named parameters are usually easier to read and maintain.

## Positional parameters

```
@Query("""
       SELECT s
       FROM Student s
       WHERE s.age >= ?1
         AND s.active = ?2
       """)
List<Student> findStudents(
        Integer minimumAge,
        Boolean active
);
```

The indexes are one-based: `?1`, `?2`, and so on.

## Relationship joins

Suppose `Student` contains:

```
@ManyToOne
private Department department;
```

An explicit JPQL join follows the entity relationship:

```
@Query("""
       SELECT s
       FROM Student s
       JOIN s.department d
       WHERE d.active = true
       """)
List<Student> findStudentsInActiveDepartments();
```

JPQL uses:

```
s.department
```

not a database expression such as:

```
s.department_id = d.id
```

Hibernate already knows the join-column mapping from the entity relationship.

## Implicit versus explicit joins

Explicit join:

```
@Query("""
       SELECT s
       FROM Student s
       JOIN s.department d
       WHERE d.name = :departmentName
       """)
List<Student> findByDepartmentName(
        @Param("departmentName") String departmentName
);
```

Path navigation with an implicit join:

```
@Query("""
       SELECT s
       FROM Student s
       WHERE s.department.name = :departmentName
       """)
List<Student> findByDepartmentName(
        @Param("departmentName") String departmentName
);
```

Both can express the same filtering requirement. An explicit join is useful when:

- The joined entity needs an alias.
- Several conditions refer to the joined entity.
- The join type must be controlled.
- The query selects data from both entities.

## Inner join

```
JOIN s.department d
```

An inner join includes a student only when a matching department exists. A student whose `department` is `null` is excluded.

## Left join

```
@Query("""
       SELECT s.name, d.name
       FROM Student s
       LEFT JOIN s.department d
       """)
List<Object[]> findStudentAndDepartmentNames();
```

A left join preserves students without a department.

```
Aditya | CSE
Rohit  | IT
Harsh  | null
```

For production code, a DTO or interface projection is usually clearer than exposing `Object[]` to higher layers.

## Static ordering in JPQL

```
@Query("""
       SELECT s
       FROM Student s
       WHERE s.active = true
       ORDER BY s.name ASC
       """)
List<Student> findActiveStudentsOrderedByName();
```

Multiple sort fields:

```
@Query("""
       SELECT s
       FROM Student s
       JOIN s.department d
       WHERE s.active = true
       ORDER BY d.name ASC, s.name ASC
       """)
List<Student> findActiveStudentsOrdered();
```

Static ordering is appropriate when the business operation always requires the same order.

### Dynamic ordering with `Sort`

```
@Query("""
       SELECT s
       FROM Student s
       WHERE s.active = true
       """)
List<Student> findActiveStudents(Sort sort);
```

```
Sort sort = Sort.by("name").ascending();

List<Student> students =
        studentRepository.findActiveStudents(sort);
```

---

# 12. Native Queries

JPQL operates on the entity model. Native SQL is useful when a query needs:

- Database-specific functions
- Vendor-specific syntax

- Common table expressions or advanced SQL features
- Existing SQL that cannot be expressed cleanly in JPQL
- Precise control over the SQL statement

## Using `@Query(nativeQuery = true)`

```
@Query(
    value = """
            SELECT *
            FROM students
            WHERE email = :email
            """,
    nativeQuery = true
)
Optional<Student> findByEmailUsingNativeQuery(
        @Param("email") String email
);
```

`nativeQuery = true` tells Spring Data to interpret the string as database SQL instead of JPQL.

Recent Spring Data JPA versions also provide `@NativeQuery`, a composed form designed specifically for native queries:

```
@NativeQuery("""
        SELECT *
        FROM students
        WHERE email = :email
        """)
Optional<Student> findByEmailUsingNativeQuery(
        @Param("email") String email
);
```

## Native named parameters

```
@Query(
    value = """
            SELECT *
            FROM students
            WHERE age >= :minimumAge
              AND active = :active
            """,
    nativeQuery = true
)
List<Student> findStudentsUsingNativeQuery(
        @Param("minimumAge") Integer minimumAge,
        @Param("active") Boolean active
);
```

## Native positional parameters

```
@Query(
    value = """
            SELECT *
            FROM students
            WHERE age >= ?1
              AND active = ?2
            """,
    nativeQuery = true
)
List<Student> findStudentsUsingNativeQuery(
        Integer minimumAge,
        Boolean active
);
```

> A native query is coupled to the database schema and possibly to the selected database vendor. Use it when the benefit outweighs the loss of portability.

# 13. Sorting

Without `ORDER BY`, a relational database does not guarantee a meaningful result order. Sorting is therefore part of query design, not just presentation.

## Creating a `Sort`

```
Sort sort = Sort.by("name");
```

Ascending order is the default. An explicit form is:

```
Sort sort = Sort.by(
        Sort.Direction.ASC,
        "name"
);
```

Use it with a repository method:

```
List<Student> students = studentRepository.findAll(sort);
```

The `Sort` argument must not be `null`. Use `Sort.unsorted()` when no ordering is required.

## Ascending and descending order

```
Sort byName = Sort.by("name").ascending();

Sort byNewest =
        Sort.by("admissionDate").descending();
```

## Multiple properties

```
Sort sort = Sort.by("department.name")
        .ascending()
```

```
        .and(Sort.by("name").ascending());
```

Conceptually:

```
ORDER BY department.name ASC,
         student.name ASC
```

Sort properties refer to entity properties, not database column names:

```
Sort.by("admissionDate")
```

## Dynamic sorting with a query method

```
List<Student> findByActiveTrue(Sort sort);
```

```
Sort sort = Sort.by(
        Sort.Order.desc("age"),
        Sort.Order.asc("name")
);

List<Student> students =
        studentRepository.findByActiveTrue(sort);
```

The predicate remains fixed while the caller controls the order.

## Validating user-controlled sorting

Suppose an API accepts:

```
GET /students?sortBy=name&direction=asc
```

Do not pass arbitrary client input directly into `Sort`. Validate it against an allowlist:

```
private static final Set<String> ALLOWED_SORT_FIELDS =
        Set.of("name", "age", "admissionDate");

private Sort createSort(
        String property,
        String direction
) {
    if (!ALLOWED_SORT_FIELDS.contains(property)) {
        throw new IllegalArgumentException(
                "Unsupported sort property: " + property
        );
    }

    Sort.Direction sortDirection =
            Sort.Direction.fromString(direction);

    return Sort.by(sortDirection, property);
}
```

Validation prevents invalid property paths and keeps the API contract intentional.

# 14. Pagination

## Why pagination exists

Suppose a table contains two million students. Calling:

```
studentRepository.findAll();
```

can cause problems throughout the stack:

- The database reads too many rows.
- The network transfers too much data.
- Hibernate creates too many entity objects.

- The persistence context consumes excessive memory.
- The JSON response becomes enormous.
- The client cannot use the entire result effectively.

Pagination changes the request from:

```
Give me every student.
```

to:

```
Give me a controlled window of students.
```

## Page number and page size

With zero-based page indexing and a page size of 10:

```
Page 0 → records 1–10
Page 1 → records 11–20
Page 2 → records 21–30
```

The conceptual offset is:

```
offset = page number × page size
```

For page `2` with size `10`:

```
offset = 2 × 10 = 20
```

Conceptual SQL:

```sql
SELECT *
FROM students
```

```
ORDER BY id
LIMIT 10 OFFSET 20;
```

The actual pagination syntax is database-dependent and generated by Hibernate.

## Pageable

`Pageable` describes:

- Requested page number
- Requested page size
- Sort information
- Whether the request is paged or unpaged

```
Page<Student> findByActiveTrue(Pageable pageable);
```

`Pageable` is an interface. `PageRequest` is its commonly used implementation.

## PageRequest

Unsorted request:

```
Pageable pageable = PageRequest.of(0, 10);
```

Sorted request:

```
Pageable pageable = PageRequest.of(
        0,
        10,
        Sort.by("name").ascending()
);
```

Convenience form:

```
Pageable pageable = PageRequest.of(
        0,
```

```
        10,
        Sort.Direction.ASC,
        "name"
);
```

`PageRequest.of()` requires:

- A non-negative page number
- A page size greater than zero

The last page may contain fewer elements:

```
Total elements = 45
Page size      = 20

Page 0 → 20 elements
Page 1 → 20 elements
Page 2 → 5 elements
```

At the API boundary, validate the page size and enforce a maximum value to prevent clients from requesting an unreasonably large page.

## Sorting and pagination must work together

```
Pageable pageable = PageRequest.of(
        0,
        10,
        Sort.by("name").ascending()
);
```

This request contains:

```
Page number = 0
Page size   = 10
Sort        = name ascending
```

A stable and deterministic order is important because page boundaries depend on the ordering. If the primary sort field is not unique, adding a unique tie-breaker such as `id` can prevent inconsistent ordering between requests.

```
Sort sort = Sort.by("name").ascending()
        .and(Sort.by("id").ascending());
```



## 15. `Page<T>`

```
Page<Student> page = studentRepository.findAll(
        PageRequest.of(0, 10)
);
```

A `Page<Student>` contains both the current content and pagination metadata.

### Common `Page` methods

```
List<Student> content = page.getContent();

int pageNumber = page.getNumber();
int pageSize = page.getSize();
int elementsInCurrentPage = page.getNumberOfElements();

long totalElements = page.getTotalElements();
int totalPages = page.getTotalPages();

boolean first = page.isFirst();
boolean last = page.isLast();
boolean hasNext = page.hasNext();
boolean hasPrevious = page.hasPrevious();

Sort sort = page.getSort();
```

The content contains only the entities in the current page. For a page size of 10, its size can be between 0 and 10.

`Page` extends `Slice` and adds:

- Total number of matching elements
- Total number of pages

---

# 16. Count Queries

Suppose the content query returns ten rows. Those ten rows alone cannot reveal whether the complete result contains 10, 11, 1,000, or 10 million rows.

To calculate `totalElements` and `totalPages`, a paged operation commonly needs two queries.

## Content query

```
SELECT ...
FROM students
WHERE active = true
ORDER BY name
LIMIT 10 OFFSET 20;
```

## Count query

```
SELECT COUNT(*)
FROM students
WHERE active = true;
```

For a simple derived method, Spring Data can derive both queries:

```
Page<Student> findByActiveTrue(Pageable pageable);
```

## Custom count query with `@Query`

Complex JPQL can provide an explicit count query:

```
@Query(
    value = """
            SELECT s
            FROM Student s
            JOIN s.department d
            WHERE s.active = true
              AND d.name = :departmentName
            """,
    countQuery = """
            SELECT COUNT(s)
            FROM Student s
            JOIN s.department d
            WHERE s.active = true
              AND d.name = :departmentName
            """
)
Page<Student> findActiveStudentsByDepartment(
        @Param("departmentName") String departmentName,
        Pageable pageable
);
```

An explicit count query is particularly useful when the content query contains:

- Complex joins
- Grouping
- Projections
- Fetch joins
- Native SQL

The count query should reproduce the filtering logic but omit unnecessary ordering and fetching.

---

# 17. `Slice<T>`

A `Slice` represents the current chunk of data and whether another chunk is available.

```
Slice<Student> findByActiveTrue(Pageable pageable);
```

## Common `Slice` methods

```
List<Student> content = slice.getContent();

int pageNumber = slice.getNumber();
int pageSize = slice.getSize();
int elementsInCurrentSlice = slice.getNumberOfElements();

boolean first = slice.isFirst();
boolean last = slice.isLast();
boolean hasNext = slice.hasNext();
boolean hasPrevious = slice.hasPrevious();
```

It does not provide:

```
getTotalElements()
getTotalPages()
```

## `Page` versus `Slice`

| Requirement | `Page<T>` | `Slice<T>` |
|---|---|---|
| Current content | Yes | Yes |
| Page number and size | Yes | Yes |
| Has next/previous | Yes | Yes |
| Total elements | Yes | No |
| Total pages | Yes | No |

| Requirement | `Page<T>` | `Slice<T>` |
|---|---|---|
| Count query commonly required | Yes | No |

`Slice` can be more efficient when the client only needs to know whether more data is available.

```
Page
  → limited content query
  → often an additional count query

Slice
  → limited query
  → commonly fetches one extra row to determine hasNext
```

Use `Page` when the UI needs information such as “page 3 of 25.” Use `Slice` for experiences such as “load more” or infinite scrolling, where the exact total is unnecessary.

---

# 18. Complete Mental Model

When the application calls an inherited method:

```
studentRepository.findById(1L)
        ↓
Repository proxy receives the call
        ↓
SimpleJpaRepository handles the method
        ↓
EntityManager performs the JPA operation
        ↓
Hibernate checks the persistence context
        ↓
Hibernate executes SQL if required
        ↓
Database returns the result
```

When the application calls a derived query method:

```
studentRepository.findByEmail(email)
        ↓
```

```
Repository proxy receives the call
        ↓
Prepared repository query is selected
        ↓
Method arguments are bound
        ↓
EntityManager executes the query
        ↓
Hibernate generates and executes SQL
        ↓
Results are mapped to entities
```

Spring Data JPA therefore removes repetitive repository implementation code while continuing to use the same JPA and Hibernate machinery underneath.

---

# Key Takeaways

1. JPA is a specification; Hibernate is a common JPA provider; Spring Data JPA is a repository abstraction built on top of JPA.
2. `JpaRepository<Student, Long>` captures both the entity type and its identifier type.
3. Spring creates a repository proxy at runtime; standard methods are primarily handled by `SimpleJpaRepository`.
4. `save()` chooses between `persist()` and `merge()` according to whether the entity is considered new.
5. Managed entities are updated through dirty checking, even without an explicit `save()` call.
6. Derived query methods are effective while their names remain readable.
7. JPQL works with entities and relationships; native SQL works with tables and columns.
8. Sorting should be explicit whenever order matters.
9. `Page` provides totals and commonly needs a count query; `Slice` omits totals and can be more efficient.
10. Spring Data JPA reduces boilerplate, but correct transactions, mappings, fetching, SQL design, and database performance still matter.

---